RAG Fundamentals

An Introduction to Retrieval-Augmented Generation for Cross-Functional Teams

Java | UI | DevOps | Data Science

AGENDA

The Problem

Understanding why LLMs need help and what they fail at.

The Pipeline

Breaking down the core components of RAG step-by-step.

Your Role

How Java, UI, DevOps, and DS integrate into the RAG system.

WHY RAG EXISTS

- **LLMs Hallucinate:** They sound confident even when they are completely wrong.
- **No Real-Time Knowledge:** Their training data has a strict cutoff date.
- **No Private Data:** They don't know your company's internal wikis, code, or policies.

> *"LLMs are incredibly smart... but they take exams completely closed-book."*

WHY NOT FINE-TUNE?

🔨Fine-Tuning

- **Expensive:** Requires significant GPU compute.
- **Static:** Once trained, the knowledge is baked in. Hard to update daily.
- **Concept:** Modifying the model's internal memory.
- **Best For:** Teaching an LLM a new tone, format, or language.

🗄️RAG (Retrieval-Augmented)

- **Cost-Effective:** Only costs API tokens for search/generation.
- **Dynamic:** Instantly update knowledge by adding docs to a DB.
- **Concept:** Giving the model an external memory drive.
- **Best For:** Question-answering over custom, factual data.

WHAT IS RAG?

Search First, Generate Second

Retrieval-Augmented Generation (RAG) is a technique that combines an information retrieval system with a generative AI model.

Instead of guessing, the system **searches** your private database for relevant facts, and then **generates** a final answer based strictly on what it found.

RAG PIPELINE OVERVIEW

1. Query

User asks a specific question via UI.

2. Embed

Question converts to numerical vectors.

3. Retrieve

Vector DB finds similar data chunks.

4. Prompt

Combine context + query for the LLM.

5. Generate

LLM streams final synthesized answer.

Visualizing the Flow

The magic happens in the middle. The LLM isn't acting as a knowledge base; it's acting as a **reasoning engine** processing the context provided by the Vector DB.

CORE COMPONENTS

Embeddings

Captures semantic meaning (Data Science focus).

Vector Database

The specialized search engine (DevOps/Backend).

Retriever

Logic for top-k selection and ranking (Backend focus).

LLM

Generates final readable answer (Full Stack/UI).

WHAT ARE EMBEDDINGS?

The Intuition

Computers don't understand English, they understand numbers. Embeddings translate text into high-dimensional arrays of numbers (vectors).

The Golden Rule: Words or sentences with similar meanings are plotted closer together in this mathematical space.

Analogy: Like converting sentences into GPS coordinates. "Apple" (fruit) is close to "Banana", but far from "Apple" (company).

VECTOR SEARCH (RETRIEVAL)

Keyword Search (Old Way)

Searches for exact string matches.

- Query: "Can I work from home?"
- Database has: "Remote work policy"
- Result: **Zero matches.** The words don't overlap.

Vector Search (New Way)

Searches using Cosine Similarity (Nearest Neighbors).

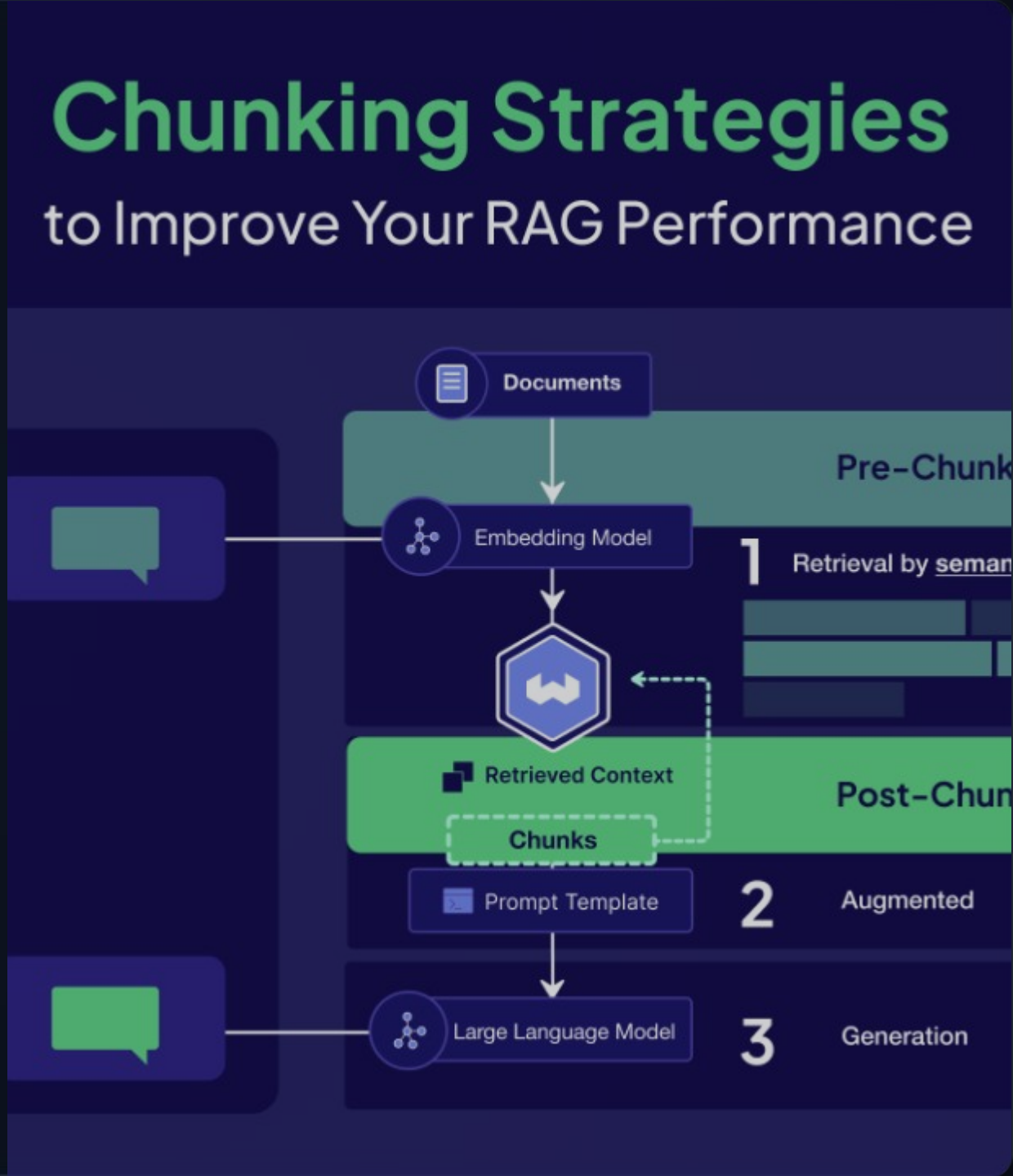
- Query: "Can I work from home?"
- Database has: "Remote work policy"
- Result: **Match found!** The vectors are mathematically close in meaning.

CHUNKING: THE HIDDEN HERO

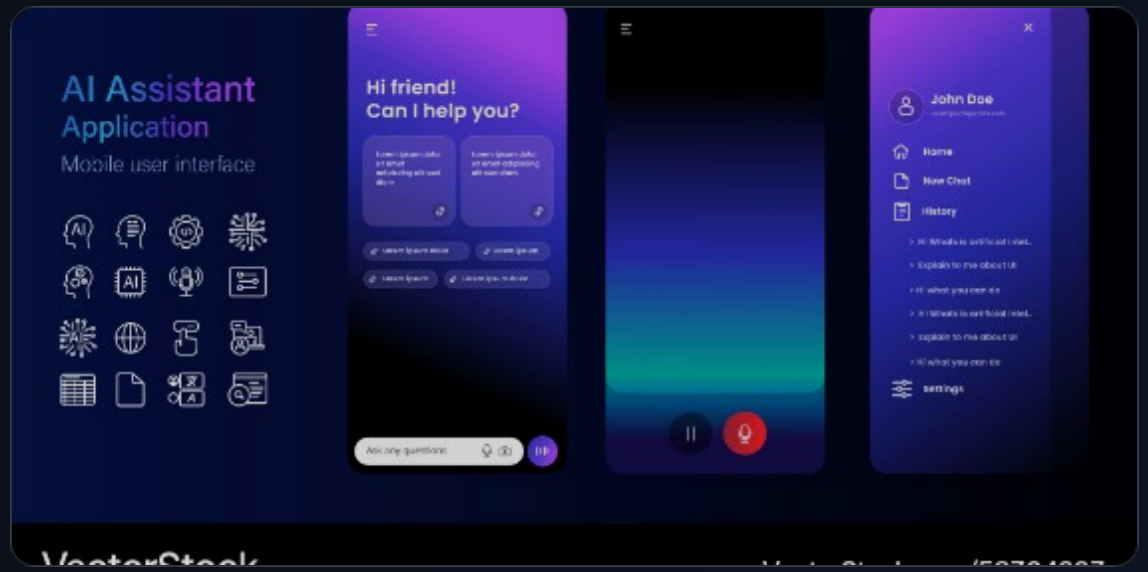
Why split data?

You can't stuff a 100-page PDF into an LLM. We must break it into paragraphs.

- **Too Small:** Lose context (e.g., "He said yes" - who is he?).
- **Too Big:** Retrieval pulls in irrelevant noise, diluting the answer.

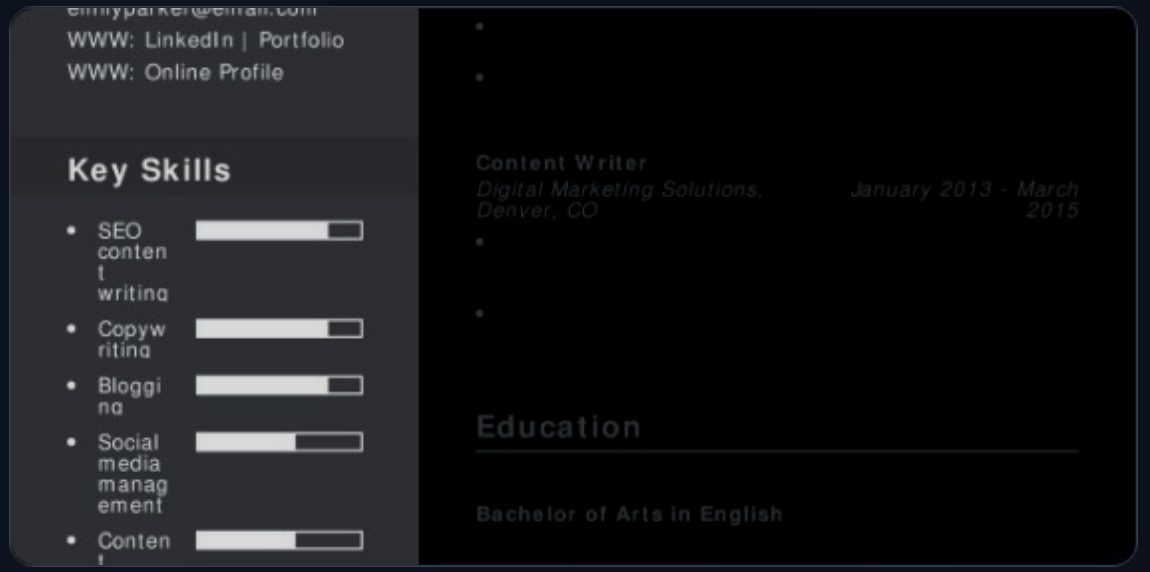


REAL-WORLD EXAMPLES



Company Policy Chatbot

Upload HR handbooks → chunk → embed → store. Employees query "Maternity leave" → retrieve exact policy clause → LLM formats response.



Automated Resume Q&A

Upload candidate CVs. Recruiter asks "Does this person know Spring Boot?" → vector search finds tech stack → LLM answers yes/no.

MINIMAL CODE IMPLEMENTATION

Simpler Than You Think

Whether in Python, Java (LangChain4j), or Node, the orchestration is usually just a few lines tying libraries together.

We avoid writing raw embedding algorithms; we use API wrappers.

```
# 1. Ingest Data
chunks = split_text(load_pdf("data.pdf"))
vector_db.insert(embed_model.embed(chunks))

# 2. Retrieve
query = "What is the vacation policy?"
context = vector_db.search(embed_model.embed(query))

# 3. Generate
prompt = f"Answer {query} using {context}"
response = llm.invoke(prompt)
print(response)
```

HOW YOUR ROLE FITS IN

Java / Backend

Building the API layer (Spring AI), managing secure DB connections, and orchestration logic.

UI / Frontend

Managing streaming chat interfaces, typing effects, and displaying citation sources clearly.

DevOps

Deploying Vector DBs (Milvus/pgvector), handling cold starts, and managing API rate limits securely.

Data Science

Optimizing embedding models, improving chunking strategies, and evaluating hallucination rates.

LIMITATIONS OF RAG

Retrieval Failure = Generation Failure:
If the Vector DB returns the wrong document chunk, the LLM will confidently give the wrong answer.

Latency Issues:
Embedding the query, searching the DB, and waiting for the LLM to generate text adds significant network overhead.

Context Window Limits:
You cannot pass infinite retrieved documents to the LLM. You must strictly limit results to top-K chunks.

ADVANCED CONCEPTS (SNEAK PEEK)

Hybrid Search

Combining old-school Keyword Search + modern Vector Search for better accuracy.

Re-ranking

Using a secondary ML model to re-order search results before sending them to the LLM.

Agents

Giving the LLM "tools" to decide when to use RAG vs when to calculate math or hit APIs.

WHEN TO USE RAG

✓Use RAG When:

- Data is dynamic and updates frequently.
- You need to query private or highly specific documents.
- You need **explainability** (the system must cite its sources).

✗Avoid RAG When:

- The task is purely reasoning (e.g., "Write a poem", "Refactor this code").
- No external factual knowledge is needed.
- The LLM already knows the subject deeply.

FINAL TAKEAWAYS

1

External Brain

RAG acts as an external memory drive for LLMs, stopping hallucinations.

2

Retrieval > Generation

The quality of your search dictates the quality of the AI's final answer.

3

System Design

Building AI isn't just modeling anymore; it's robust software engineering.

Questions?

Thank you for your attention.

 github.com/your-profile

IMAGE SOURCES

https://png.pngtree.com/png-clipart/20250103/original/pngtree-futuristic-robot-using-laptop-cartoon-vector-artwork-png-image_20059325.png

Source: pngtree.com

https://media.istockphoto.com/id/2234389834/photo/businessman-holding-magnifying-glass-focusing-on-target-with-customer-icons-digital-marketing.jpg?s=612x612&w=0&k=20&c=_S987UNkN1ZY9UWvxSqlVy3UVsD4D5IQTwAhNt6l-mg=

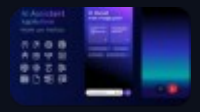
Source: www.istockphoto.com

https://www.babylonjs.com/assets/videos/clusteredLights.jpg

Source: www.babylonjs.com

https://weaviate.io/assets/images/hero-372a1930c6d24f89e7d7189207c42125.png

Source: weaviate.io



https://cdn.vectorstock.com/i/1000v/49/93/ai-chatbot-interface-dark-theme-vector-58304993.jpg

Source: www.vectorstock.com

https://www.resumebuilder.com/wp-content/uploads/2025/09/FreelanceWriter.pdf.jpeg

Source: www.resumebuilder.com

IMAGE SOURCES

https://miro.medium.com/v2/resize:fit:1400/1*rM_iP_jYQEVIfCZeXOaLZw.png

Source: medium.com